Cross-validation

Albert Y. Kim

Review of k-Nearest Neighbors

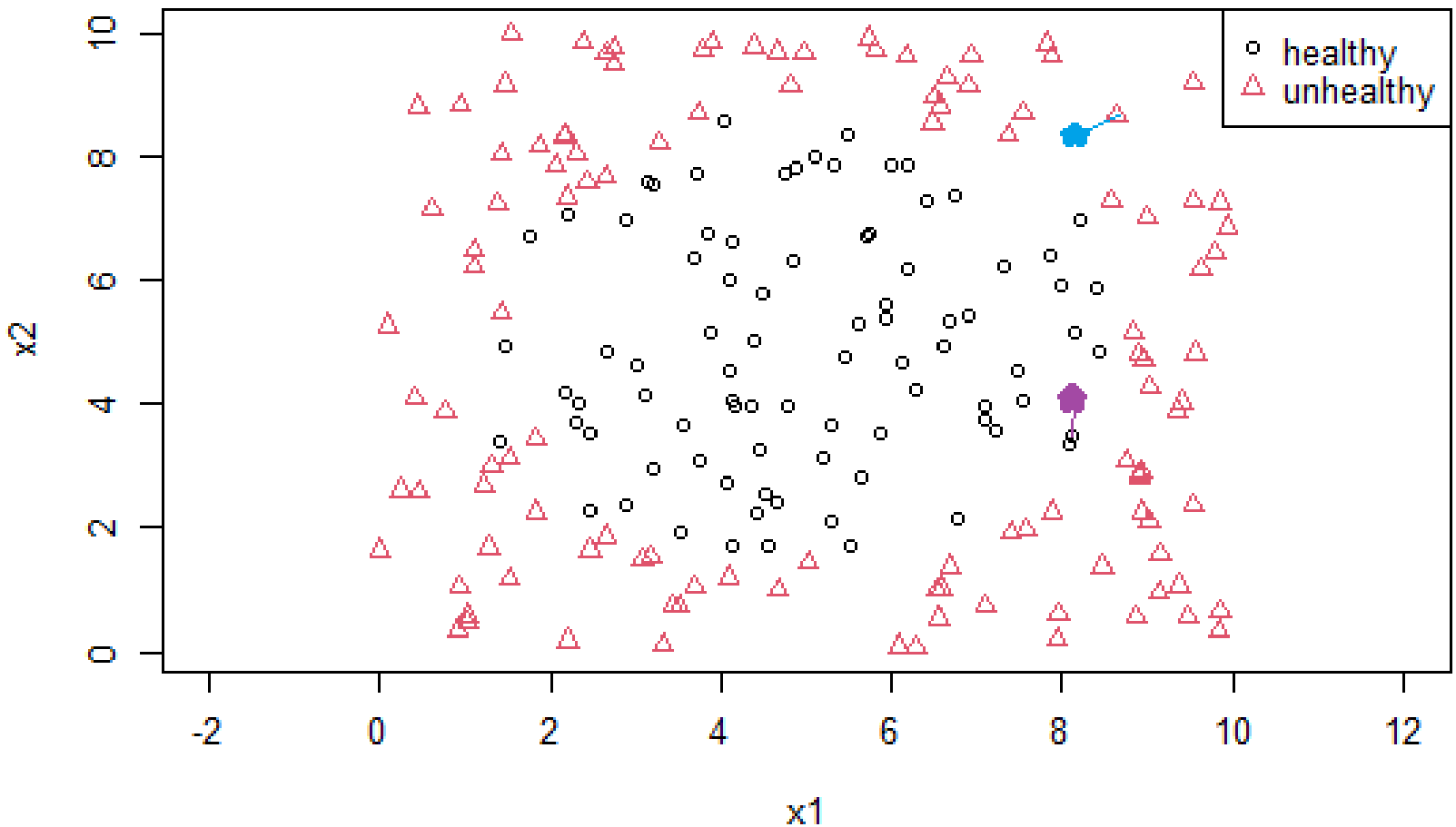
The k-nearest neighbors algorithm makes predictions by aggregating the k observed samples closest to a new data-point. In order to use the algorithm, we must provide the following:

1. A way of measuring distance
2. A value for k
3. If neighbors should contribute uniformly or be weighted by distance

A **parameter** is an unknown value that an algorithm needs to make predictions. A **tuning parameter** (or **hyperparameter**) must be specified before an algorithm is used.

Tuning Parameter: k

Recall the following toy data where we are classifying y binary: healthy vs. unhealthy.

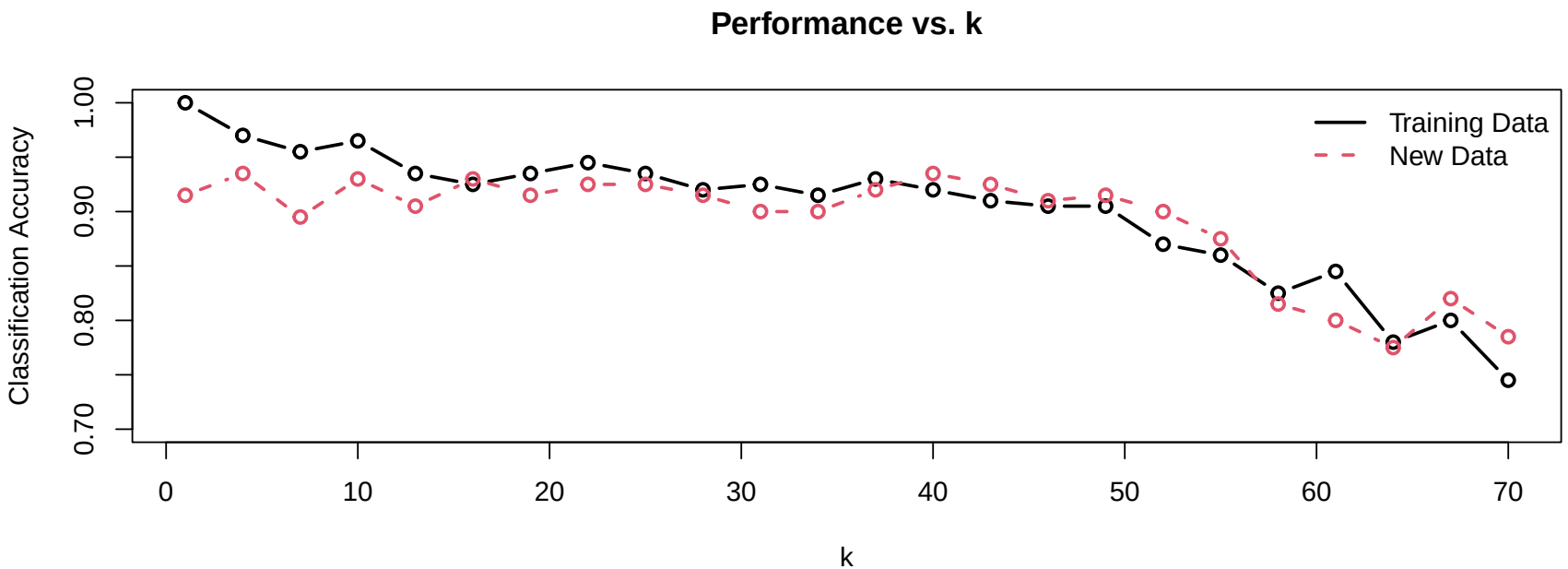


Tuning Parameter: k

Let's fit a kNN model on an $n = 200$ training data set for various values of k. Then let's plot the classification accuracy of the model

1. On that same $n = 200$ training data (black)
2. On a new $n = 200$ test set (red)

and plot the results. Note here it's *small values* of the tuning parameter k that overfit the training data.



Tuning Parameters

But what is the *optimal value* of k to use? In this case has the *largest* classification accuracy?

- We'd like a way to choose the optimal value of our tuning parameter k.
- We'd like to avoid using the test data to do so. We want to keep this data independent and separate.
- How might we determine good values for tuning parameters using *only the training data*?

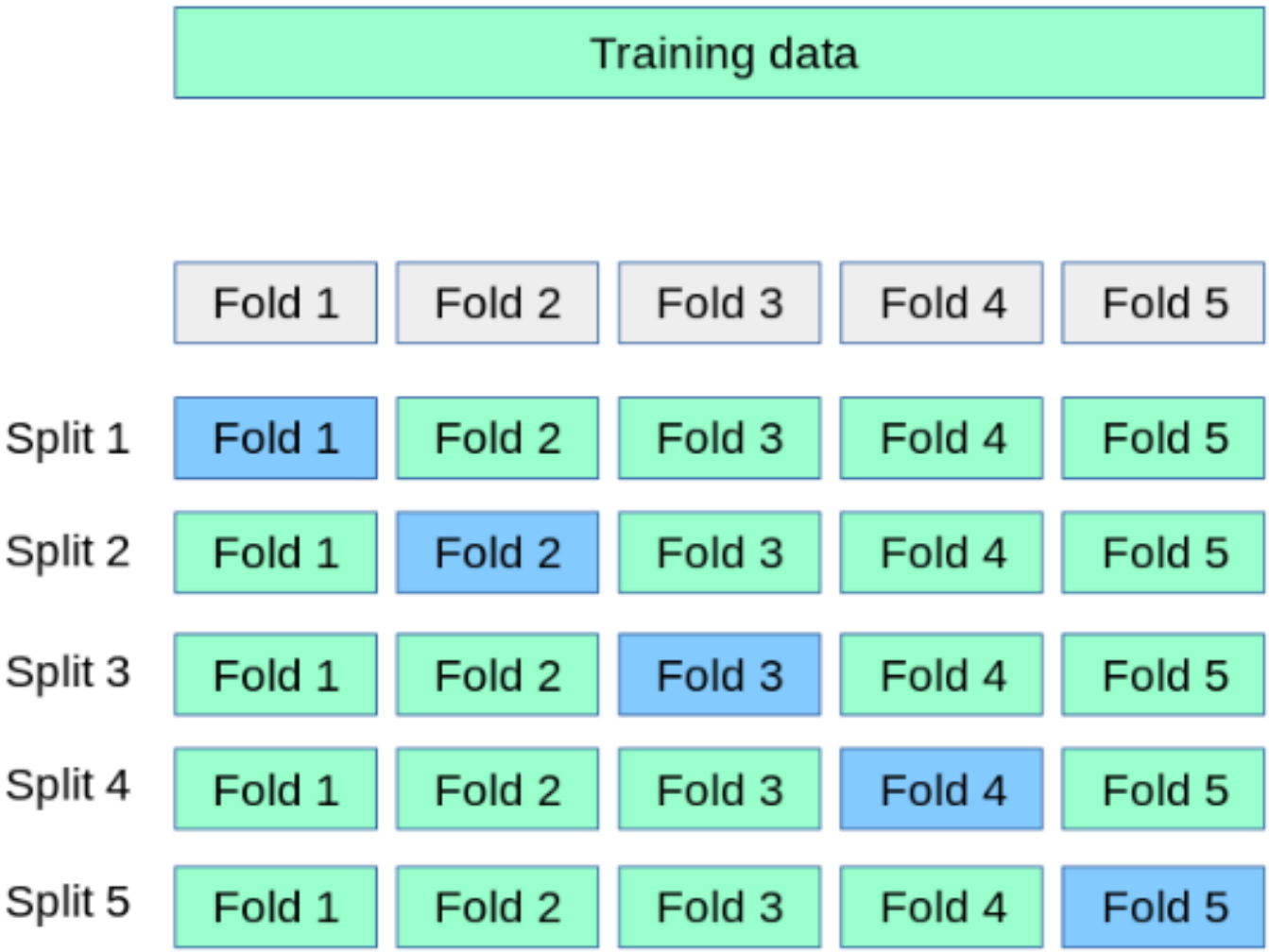
Cross-validation

Cross-validation (CV) is a general term describing methods of repeated data-splitting used to evaluate the performance of a model on data that was not used in the training process.

In other words, we are *estimating the performance of the model on new unseen data*.

k-fold Cross-validation: Visualization

Behold: k-fold crossvalidation with k=5 (not to be confused with k from kNN or k-means clustering)



k-fold Cross-validation: Pseudocode

Say you are given predictor variables X and outcome variable y

```
# Assign n obs into k folds
fold_id = sample(0:(k-1), size = n)

# save estimated errors here
fold_errors = zeros(size = k)

# Loop through each fold:
for i in range(k):
  # fit/train the model on these:
  train_X = X[fold_id != i]
  train_y = y[fold_id != i]

  # predict/evaluate model on these:
  eval_X = X[fold_id == i]
  eval_y = y[fold_id == i]

  # fit model:
  model.fit(X = train_X, y = train_y)

  # get predictions
  eval_y_hat = model.predict(X = eval_X)

  # Calculate and save error:
  fold_errors[i] = error(y_true = eval_y, y_pred = eval_y_hat)

np.mean(fold_errors)
```

k-fold Cross-validation: Simple example

- Let our data have $n = 6$ points, x has $p = 1$ dimensions, y numerical.
- The predictive model we'll use is linear regression
- Use RMSE as our measure for model error

point	x	y
1	1	2
2	2	3
3	4	5
4	5	7
5	8	9
6	9	8

Assign folds

First we randomly assign each observation to one of $k = 3$ folds...

point	x	y	fold_id
1	1	2	0
2	2	3	1
3	4	5	2
4	5	7	0
5	8	9	1
6	9	8	2

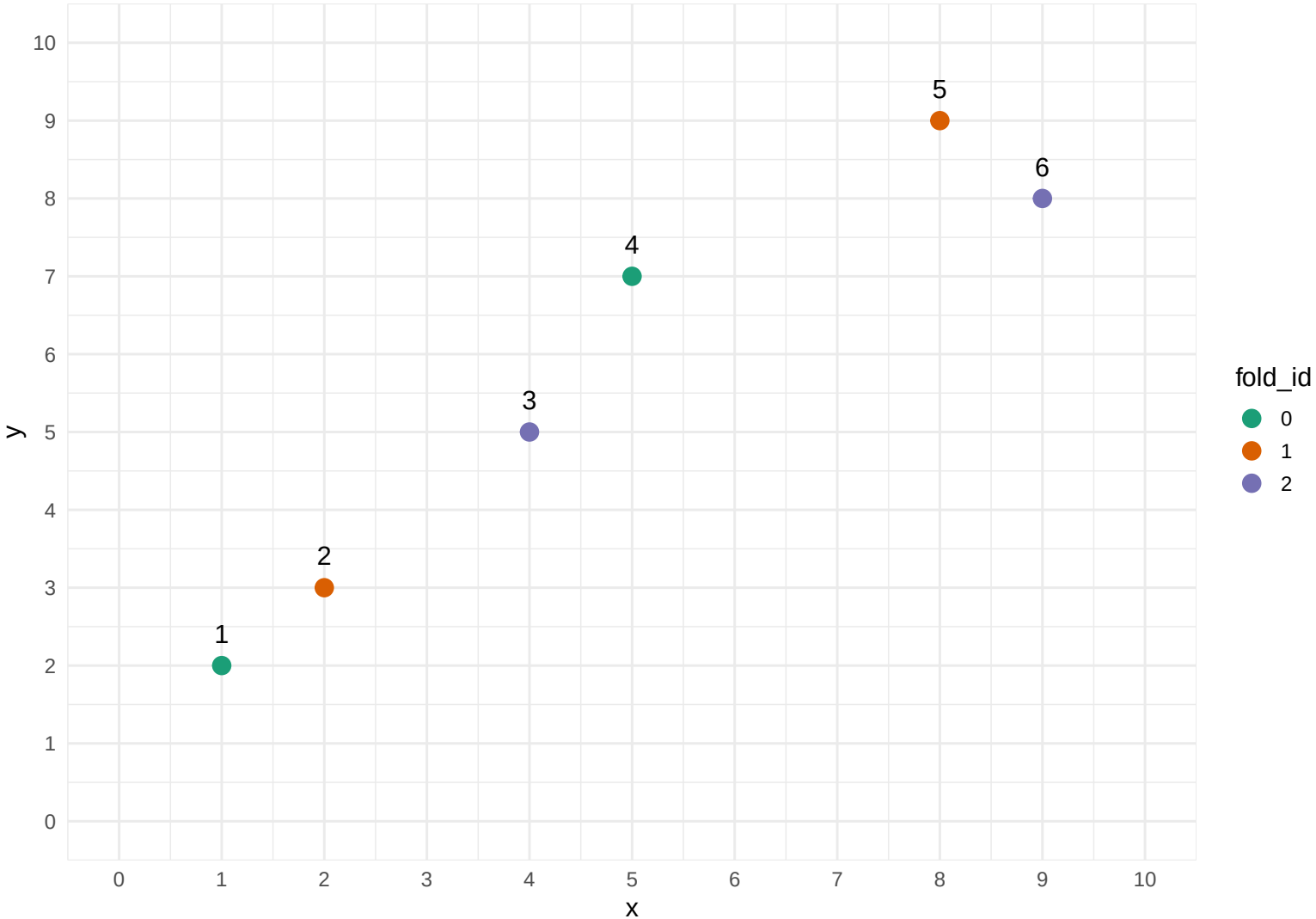
Assign folds

For visual convenience let's sort by `fold_id`...

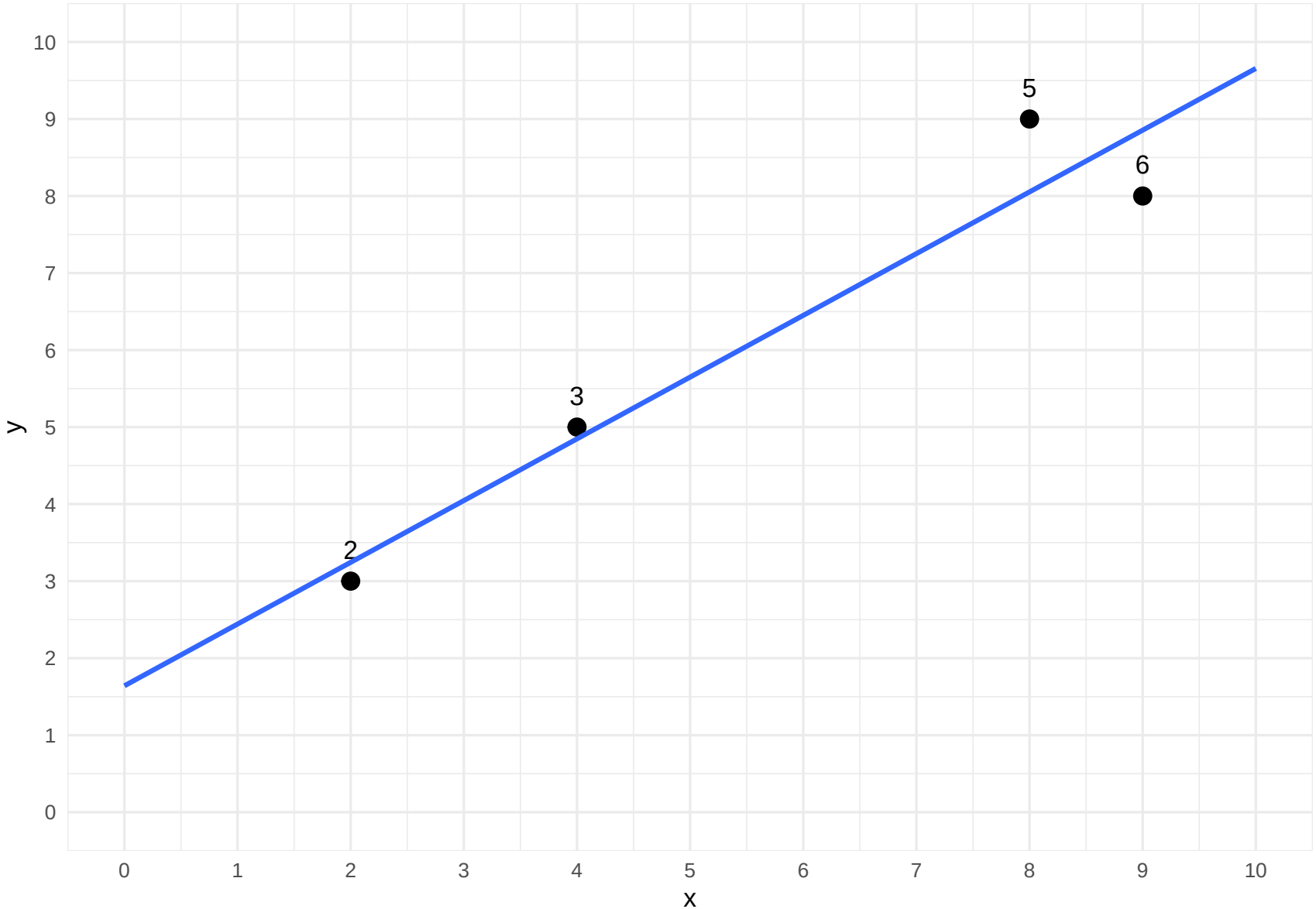
point	x	y	fold_id
1	1	2	0
4	5	7	0
2	2	3	1
5	8	9	1
3	4	5	2
6	9	8	2

Assign folds

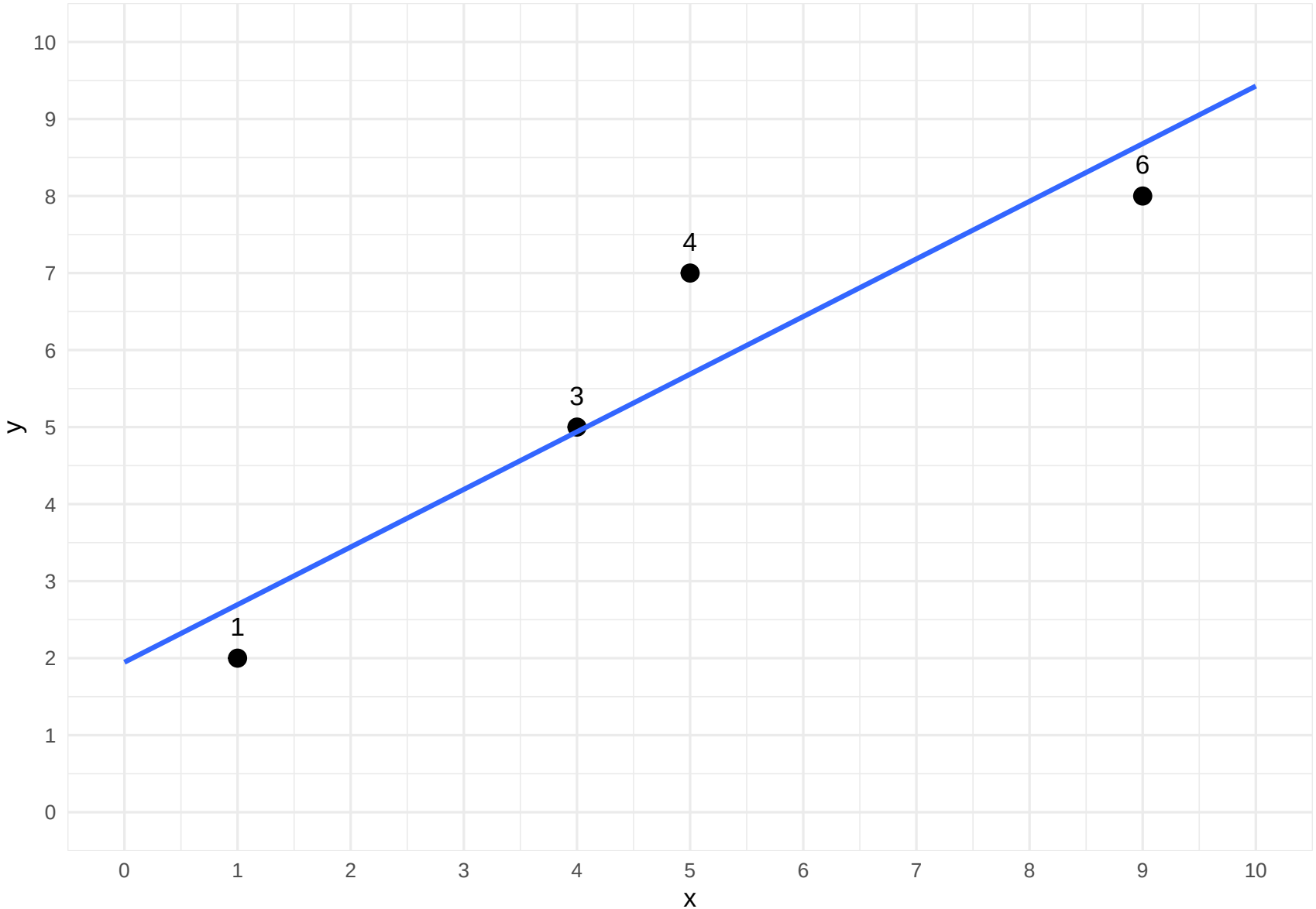
Visualize the folds. . .



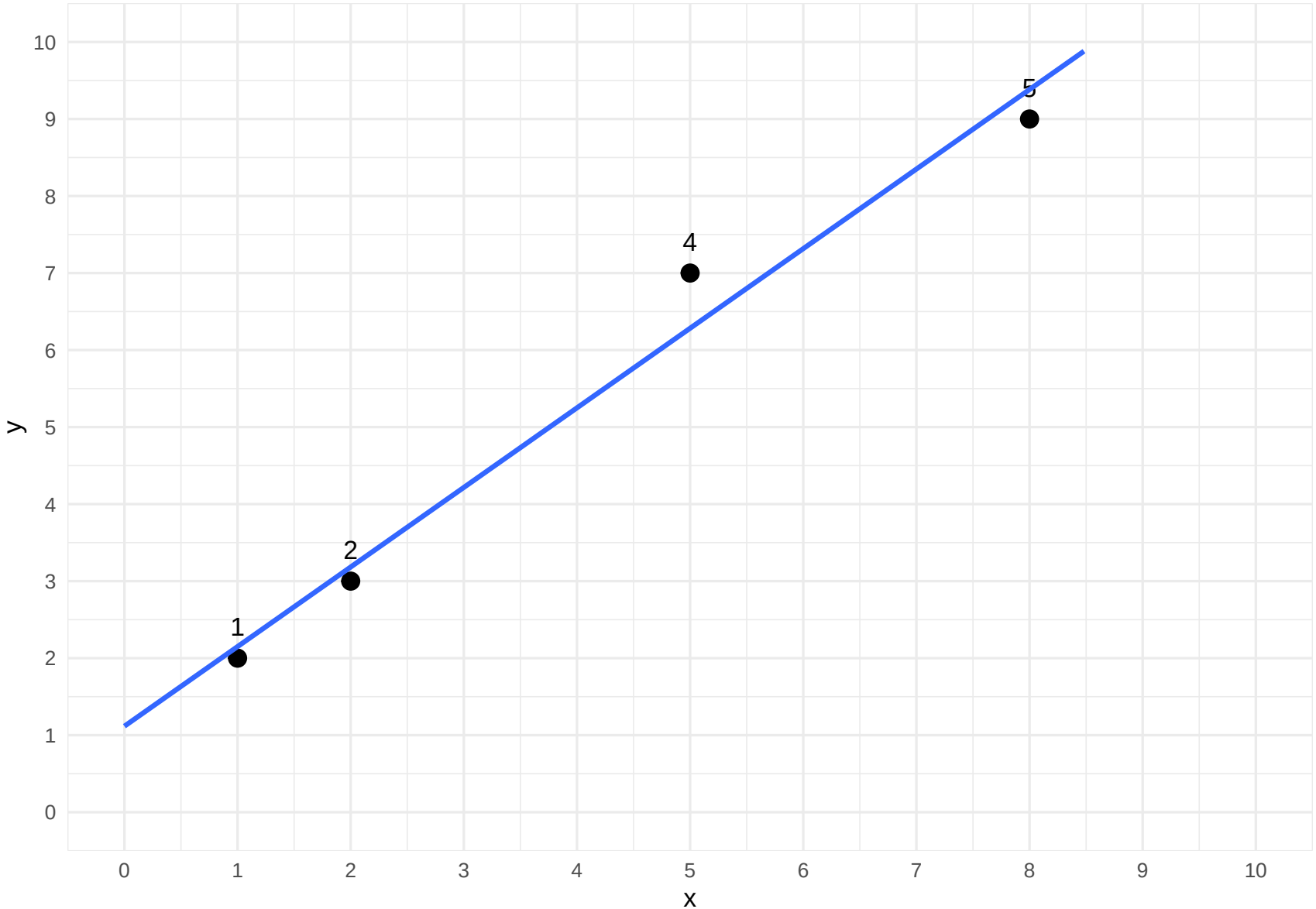
for loop: `fold_id` = 0 case



for loop: `fold_id` = 1 case



for loop: `fold_id` = 2 case

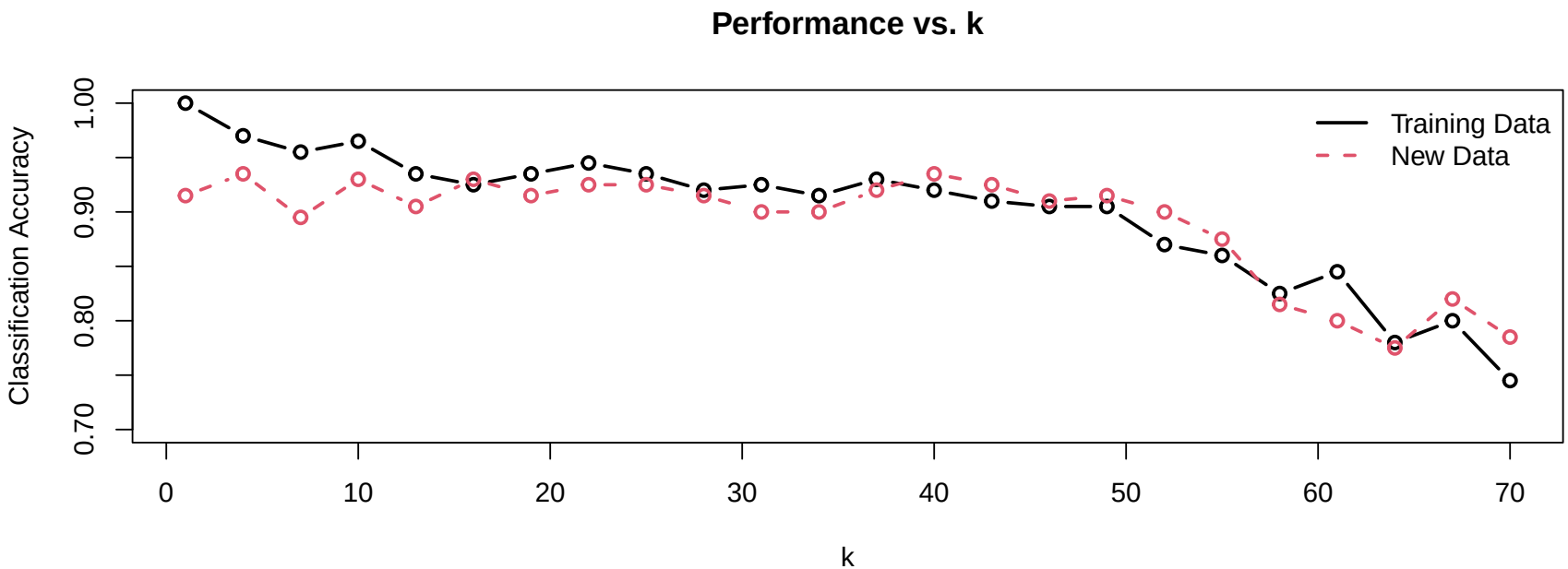


Bottom Line

The estimated RMSE error of our linear regression model on *new, unseen data* is 1.180. This estimate of the error was generated by $k = 3$ fold crossvalidation.

Back to kNN

But how does this help us find the optimal value k^* of tuning parameter value k in our kNN example?



Via **grid search**: The idea is to systematically and exhaustively search a grid spanning the parameter space

Grid Search: Pseudocode

Here let `k` be the number of folds and `k_knn` be k from kNN. Note the nested for loop

```
# Assign n obs into k folds (once, so every k_knn is compared on the same folds)
fold_id = sample(0:(k-1), size = n)

# candidate values of k_knn and save CV error of each here
k_knn_values = range(1, 50)
rmse_values = zeros(size = len(k_knn_values))

# Outer loop: through each candidate k_knn
for j in range(len(k_knn_values)):
  model = KNeighborsRegressor(n_neighbors = k_knn_values[j])
  fold_errors = zeros(size = k)

  # Inner loop - same as before: k-fold cross-validation
  for i in range(k):
    train_X = X[fold_id != i]
    train_y = y[fold_id != i]
    eval_X = X[fold_id == i]
    eval_y = y[fold_id == i]

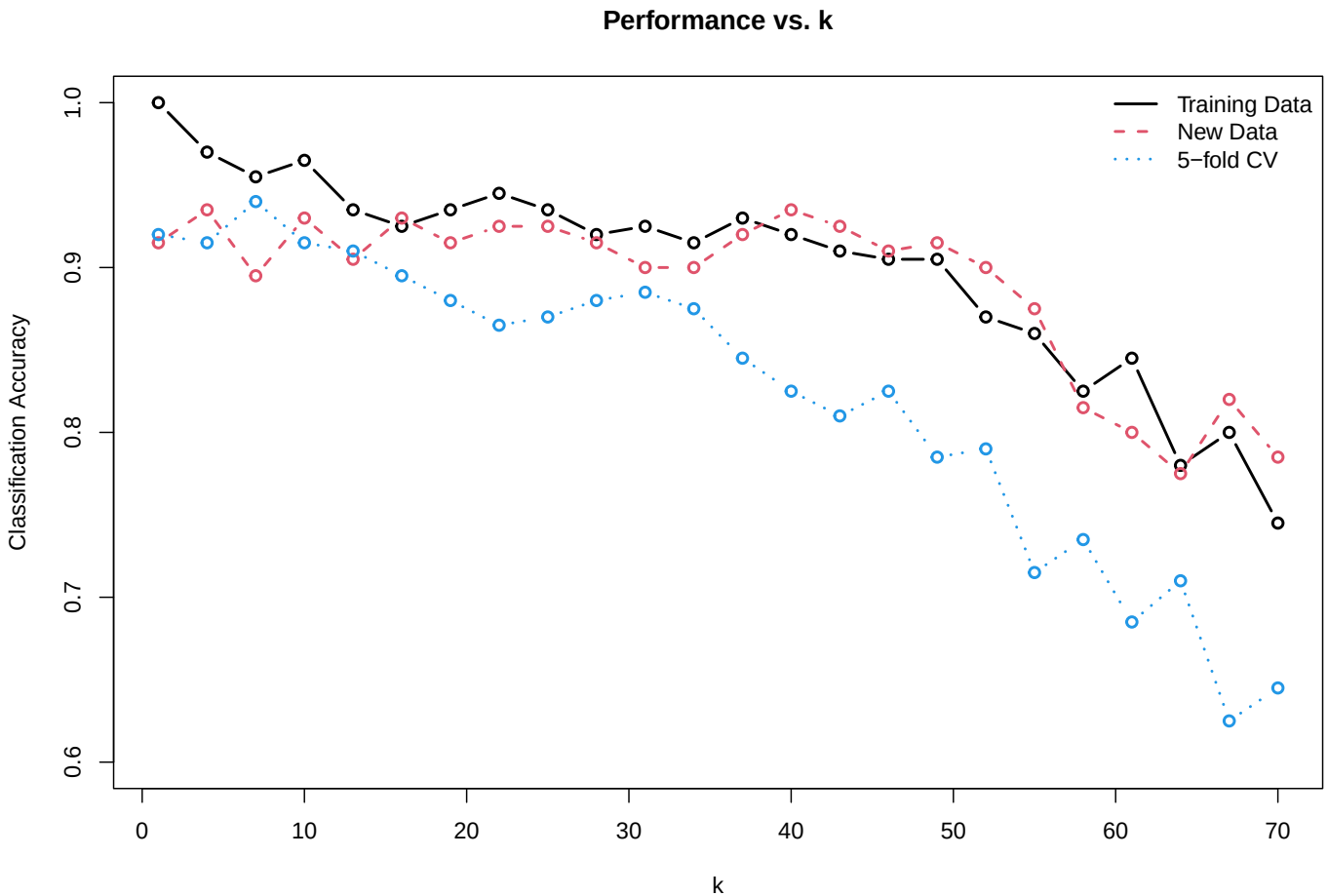
    model.fit(X = train_X, y = train_y)
    eval_y_hat = model.predict(X = eval_X)
    fold_errors[i] = error(y_true = eval_y, y_pred = eval_y_hat)

  # save CV error for this k_knn
  rmse_values[j] = np.mean(fold_errors)

# Pick the k_knn with the smallest CV error
best_k_knn = k_knn_values[np.argmin(rmse_values)]

# Refit using best k_knn on all the data
model = KNeighborsRegressor(n_neighbors = best_k_knn)
model.fit(X = X, y = y)
```

Grid Search: Back to toy data



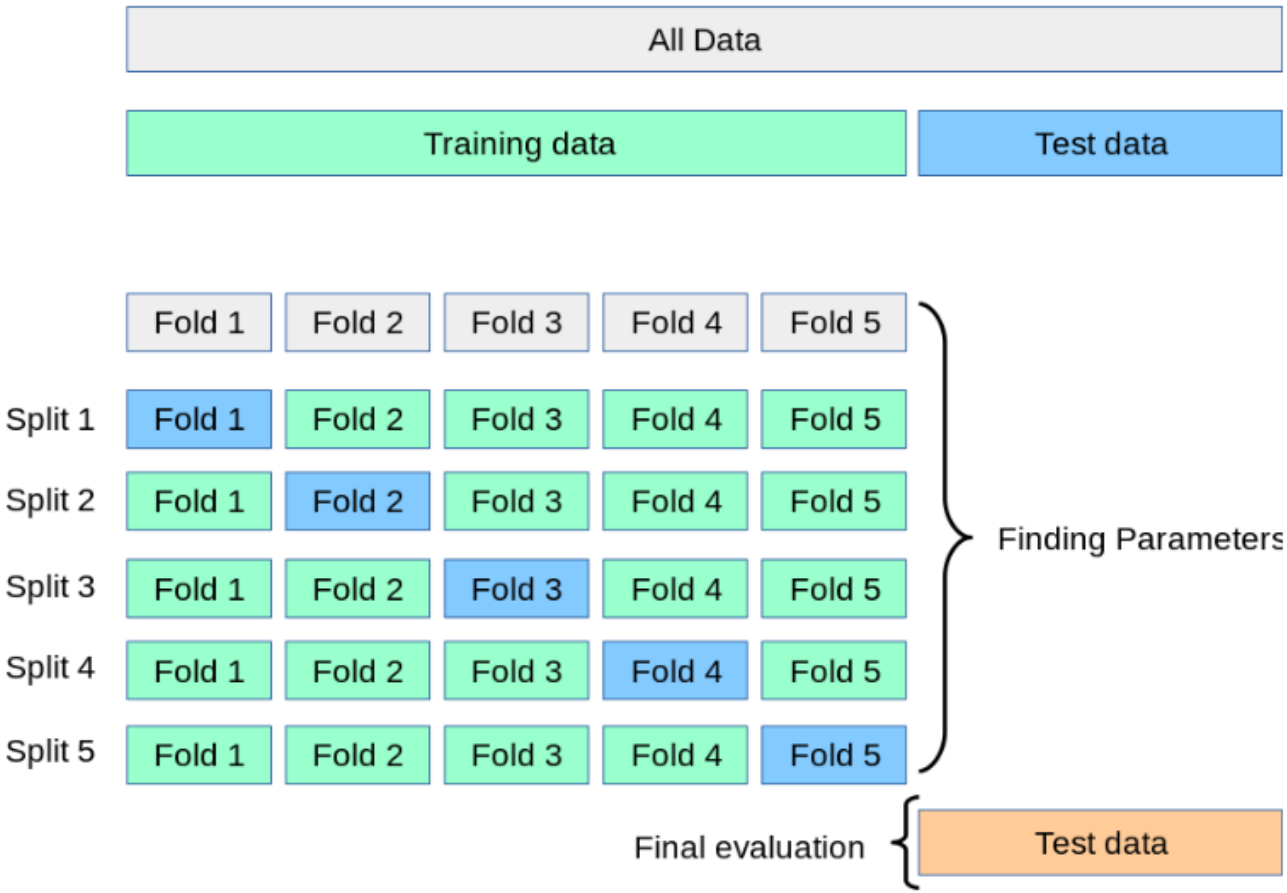
Grid Search: Expanded

We can search over more than just tuning/hyperparameters. Example: Search over 1) $k \in \{3, 4, 5\}$, 2) Euclidean or Manhattan distance, and 3) uniform or distance weighting.

k	Distance	Weight
3	euclidean	uniform
4	euclidean	uniform
5	euclidean	uniform
3	manhattan	uniform
4	manhattan	uniform
5	manhattan	uniform
3	euclidean	distance
4	euclidean	distance
5	euclidean	distance
3	manhattan	distance
4	manhattan	distance
5	manhattan	distance

We use Python to train and evaluate each row of this grid using cross-validation to reliably assess performance.

In Practice: Final step of k-fold Cross-validation



Final notes

- Unlike HW5 -> `lab05-2` on kNN -> Q1 where you did a *single* train/test split, using CV ensures every observation gets a turn at having their y be predicted via $\widehat{f}(\vec{x}) = \widehat{y}$
- For k-fold CV you fit a total of k models $\widehat{f}$
- How many folds to use?
  - In practice typically 5-fold or 10-fold CV is used
  - Special case: If $k = n$ i.e. each fold contains a single observation, this is called leave-one-out cross-validation (LOOCV)